

StockRoom

A stock control app, shipped through a full DevOps pipeline

Kartavya Panchal
Roll 24bcs10343 · DevOps Final Capstone

What I built

A warehouse stock tracker. Small on purpose: the work is in how it is tested, packaged, secured, deployed and watched.



Track items

SKU, category, quantity, unit price and a reorder level for every item



Flag low stock

Items at or below the reorder level are marked for a new order



See stock value

Totals for items, units, value and categories from /api/items/stats

6
REST endpoints

12
pytest tests

3
containers

2
replicas per tier

Tech stack

Application
FastAPI · SQLAlchemy · Alembic · PostgreSQL 16 · React 18 + Vite

Testing
pytest with an in-memory SQLite test database

Containers
Docker multi-stage builds, non-root users, Docker Compose

CI/CD
GitHub Actions · GitHub Container Registry · SHA tags

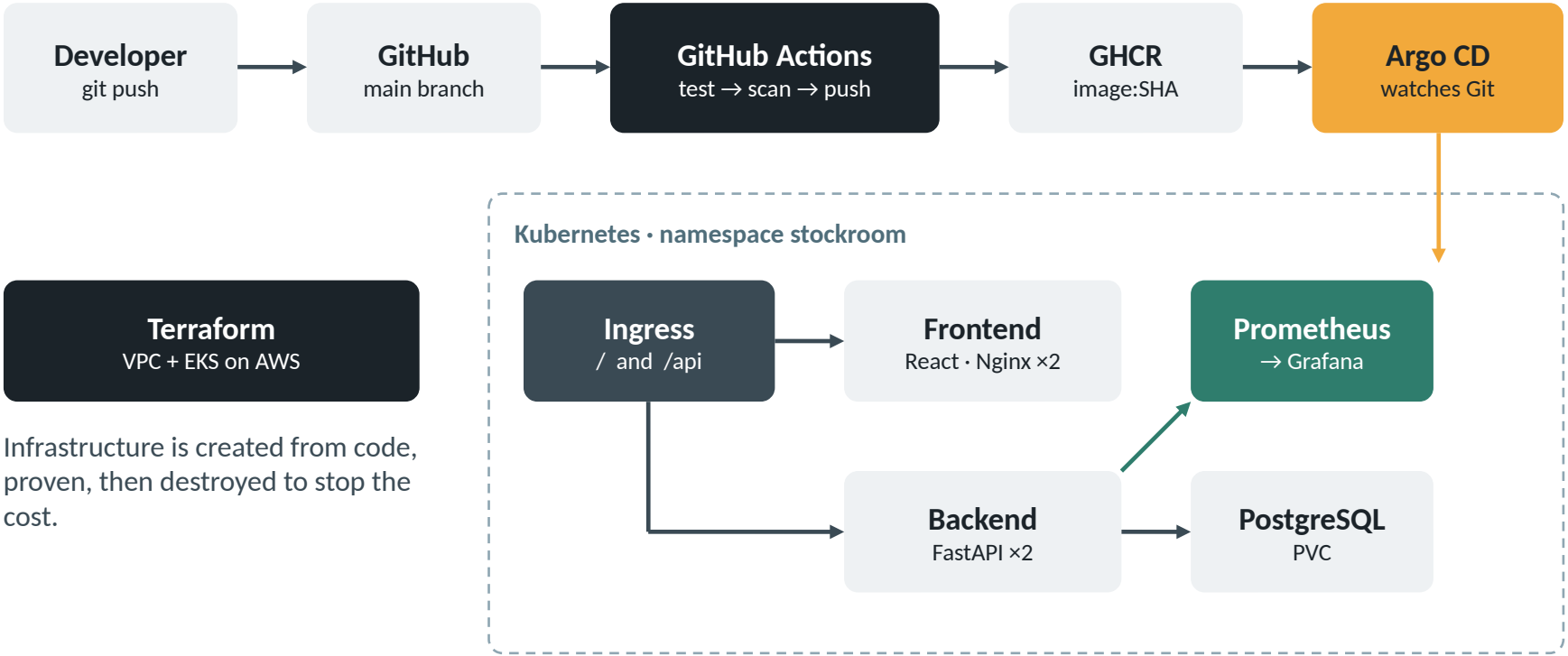
Security
Trivy 0.75 gate on fixable HIGH and CRITICAL CVEs

Infrastructure
Terraform · AWS VPC · Amazon EKS 1.36

Kubernetes
Helm · ingress-nginx · Argo CD (GitOps) · Minikube

Observability
kube-prometheus-stack: Prometheus + Grafana

Architecture



Infrastructure is created from code, proven, then destroyed to stop the cost.

CI/CD pipeline

JOB 1

Test

- pytest -v (12 tests)
- npm ci + vite build

Stops on a failing test

JOB 2

Images

- docker build ×2
- Trivy scan ×2
- push to GHCR

Stops on a fixable HIGH/CRITICAL CVE

JOB 3

GitOps

- yq writes the SHA into values.yaml
- commit with [skip ci]

One commit = one image tag. `ghcr.io/kartavya37/stockroom-backend:<full SHA>`
No "latest" tag. A rollback is a one-line revert in values.yaml.

Security gate: what Trivy found

Backend image

20 → 0

fixable HIGH / CRITICAL CVEs

Frontend image

43 → 0

fixable HIGH / CRITICAL CVEs

CVE-2026-13221 · CRITICAL

perl-base 5.40.1-6 in the Debian base image

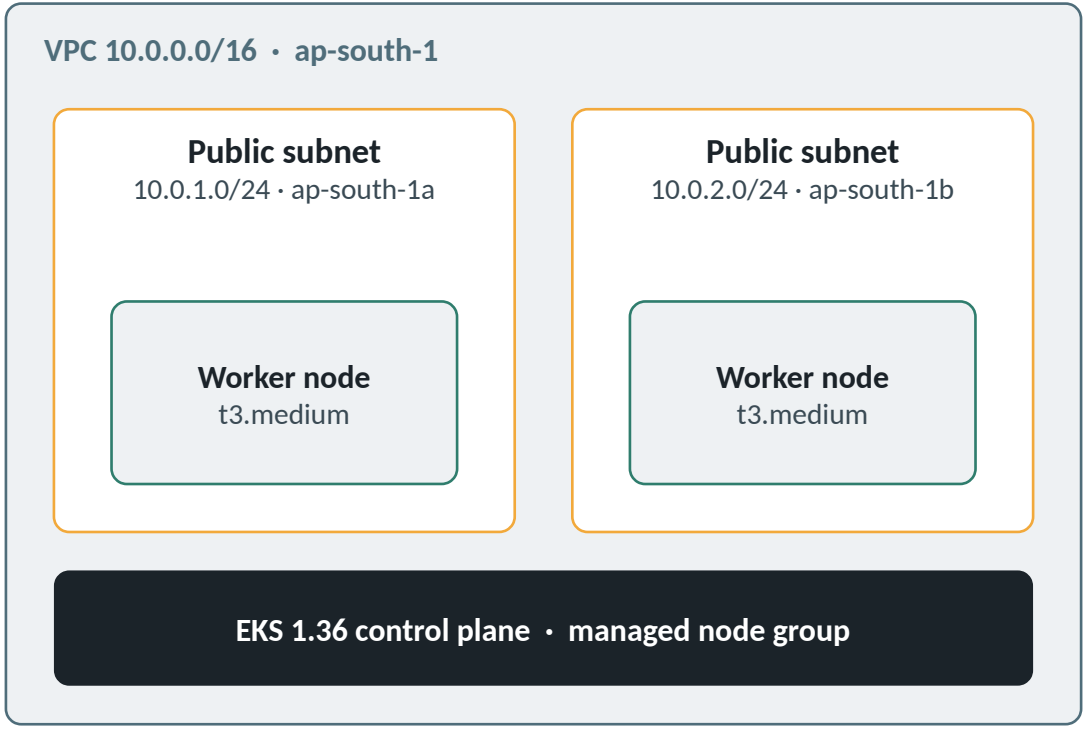
Perl mishandles very large regular expressions. StockRoom never runs Perl, so the API cannot reach it — but the package ships in the image, and Trivy reports what is present, not what is called.

Fix: apt-get upgrade → 5.40.1-6+deb13u1

The fixes

- OS upgrades in both images
- FastAPI 0.143.2 → Starlette 1.8.0
- nginx 1.27 → 1.30 Alpine

Infrastructure as code on AWS



terraform plan
shows every resource before anything is created

EKS 1.36, not 1.31
1.31–1.33 are in extended support: 6× the control-plane price

No NAT gateway
nodes sit in the public subnets: cheaper and faster to create

terraform destroy
right after the demo; no keys in the repo

Kubernetes, Helm and GitOps

One Helm chart

Deployments
backend ×2 · frontend ×2 · postgres

Services
ClusterIP for each tier

Ingress
/ → frontend, /api → backend

Secret + PVC
database URL and storage

ServiceMonitor
Prometheus scrape config

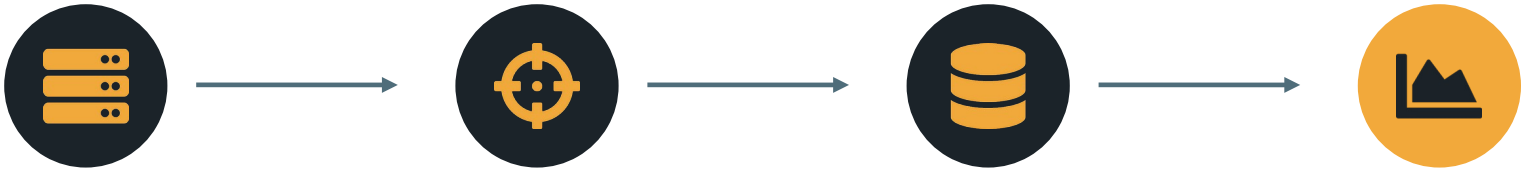
Dashboard ConfigMap
loaded by the Grafana sidecar

GitOps flow

1. git push
2. pipeline builds and scans
3. job 3 commits the new SHA
4. Argo CD detects the change
5. new Pods roll out

Auto sync, prune and self-heal. Git is the only way to change the cluster.

Observability



Backend
/metrics on every Pod

ServiceMonitor
tells Prometheus what to scrape

Prometheus
scrapes every 15 s

Grafana
StockRoom API dashboard

Dashboard panels — loaded automatically, no manual import

Requests / s by route	p95 latency	5xx error rate	Backend Pods up	CPU per Pod

Key learnings



Gates catch problems early

Trivy blocked 63 fixable CVEs before a single image reached the registry.



Test the failure, not only the fix

Two Pods racing on the migration crashed one in 5 of 5 runs. An advisory lock: 0 of 5.



SHA tags give traceability

Every running Pod points to one commit. Rollback is a one-line revert.



Git is the source of truth

With Argo CD, a push is the deployment. Cost is a design input too: EKS 1.36, not 1.31.